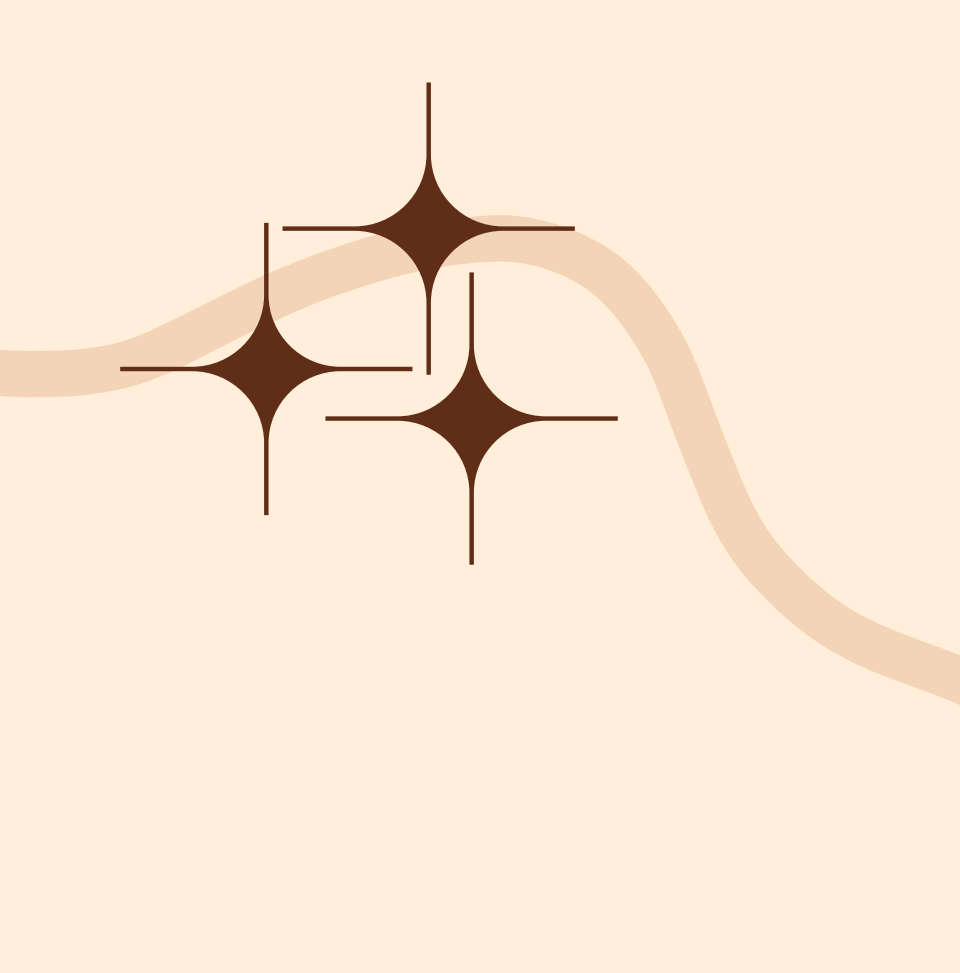




PROJECT

HANGMAN GAME



Mayank Gaikwad

1262251650

Shashant Vairamuthu

1262250804

Paulson Fernandes

1262252416

Preetam

1262251544

Shreya Khandade

1262253082

Saransh Gupta

1262251320



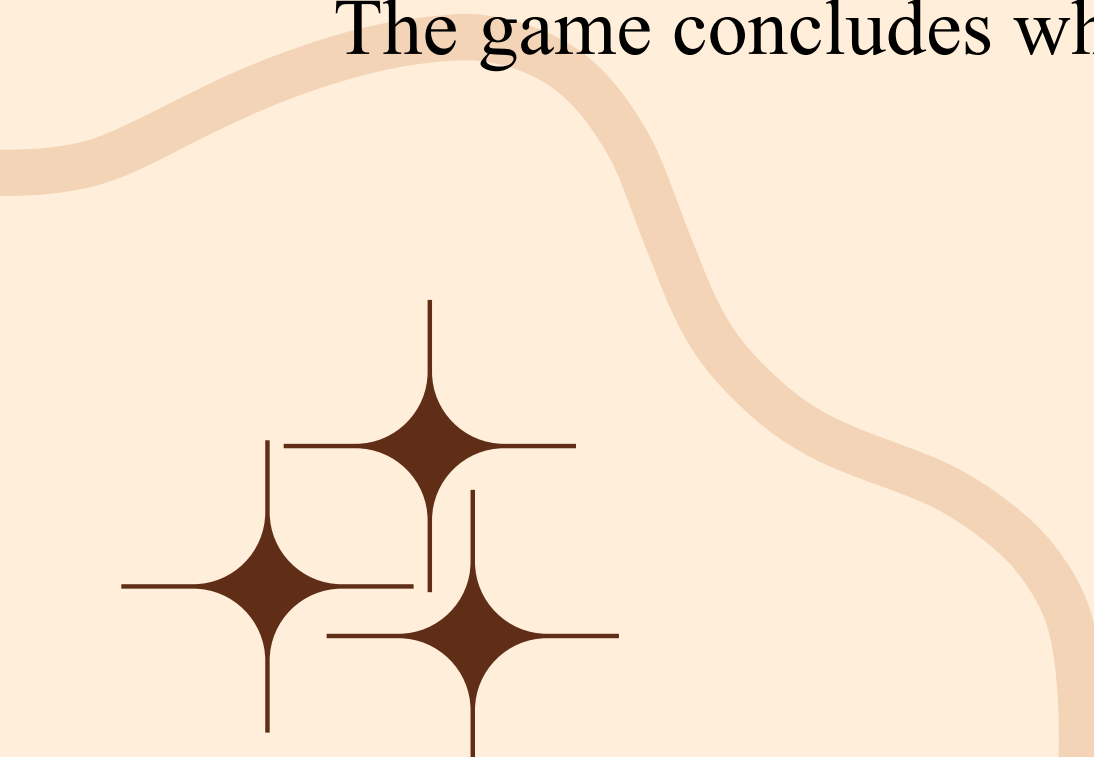
INTRODUCTION



This project presents the development of a console-based Hangman game using the C programming language. The application demonstrates the practical implementation of fundamental programming concepts, including string manipulation, control structures, and modular function design.

The program allows a user to enter a secret word, which another player attempts to guess letter by letter. The system provides continuous feedback, tracks previously guessed letters, and visually represents incorrect attempts through an ASCII-based Hangman illustration.

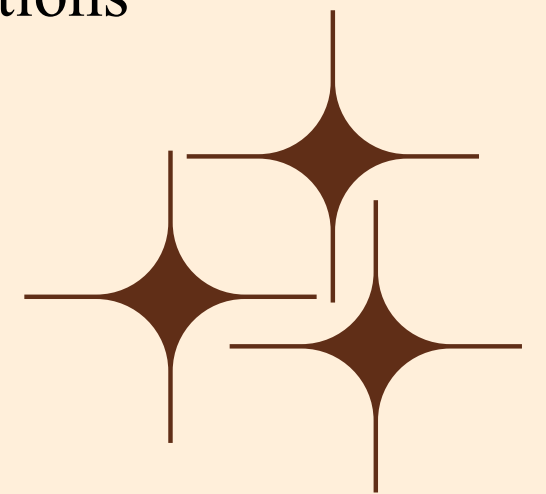
The game concludes when the player either successfully guesses the complete word or exhausts all available lives.



PROJECT OBJECTIVES

The primary objectives of this project are:

- To apply fundamental programming concepts, including:
 - Loops for game repetition and control flow
 - Conditional statements for decision-making
 - Arrays for storing words and guessed letters
- To develop knowledge in string manipulation, including comparing strings, calculating string length, and searching for characters within a string.
- To practice character handling techniques, such as converting user input to uppercase for consistency and accuracy.
- To implement modular programming principles by dividing the program into reusable functions (e.g., screen clearing, hangman drawing and the gameplay loop).



TOOLS AND TECHNOLOGIES



Technologies Used

- Programming Language: C
- Development Environment: (VS Code GCC /Sublime Text MSVC)
- Standard Libraries Used:
 - `stdio.h`
 - `string.h`
 - `ctype.h`
 - `stdlib.h`

CODE EXPLANATION

String Handling

- `strlen()` to calculate word length
- `strcmp()` to compare guessed word with secret word
- `strchr()` to check if a letter exists in a string

Character Handling

- `toupper()` ensures case consistency

Game Logic

- Loop continues while `lives > 0` and Win Condition is false
- Conditional statements determine to correct/incorrect guesses
- Dynamically updating ASCII Art

ALGORITHM

Step 1: Start

Step 2: Declare and initialize variables

secret_word (character array)

guess_word (character array)

guessed_letters (character array, initially empty)

lives = 6

message = "Enter a letter to guess (0 to Exit)."

Step 3: Clear screen

Step 4: Get secret word

if input == 0

Exit the program

Convert letters to uppercase

ALGORITHM

Step 5: Prepare guessed word

- Find length of 'secret_word'

- Fill 'guess_word' with '_' for each letter

- Terminate 'guess_word' with null character

Step 6: Start game using while loop:

- 'lives > 0' AND

- 'guess_word' is not equal to 'secret_word'

Step 7: Display game status

- Inside loop:

 - Clear screen

 - Draw hangman according to 'lives'

 - Display:

 - Current 'guess_word'

 - Remaining 'lives'

 - All 'guessed_letters'

 - Current 'message'

ALGORITHM

Step 8: Ask Player 2 to enter a guess

- Use 'scanf' to get first character input

- Use 'getchar()' to consume the \n

- Convert 'input' to uppercase

Step 9: Exit condition

- if 'input == 0'

 - Exit the program

Step 10: Validate input

- if input already exists in 'guessed_letters'

 - Update 'message' with warning

- else

 - Add 'input' to 'guessed_letters'

ALGORITHM

Step 11: Check guess

if input exists in 'secret_word'

Update 'guess_word' at all matching positions

Update 'message' as correct guess

else

Decrease 'lives' by 1

Update 'message' as wrong guess

Step 12: Repeat loop (go to Step 6)

Step 13: After loop ends, clear screen and draw final hangman

if lives > 0

Display "YOU WIN" and winner ASCII art

Show full word

else

Display "GAME OVER"

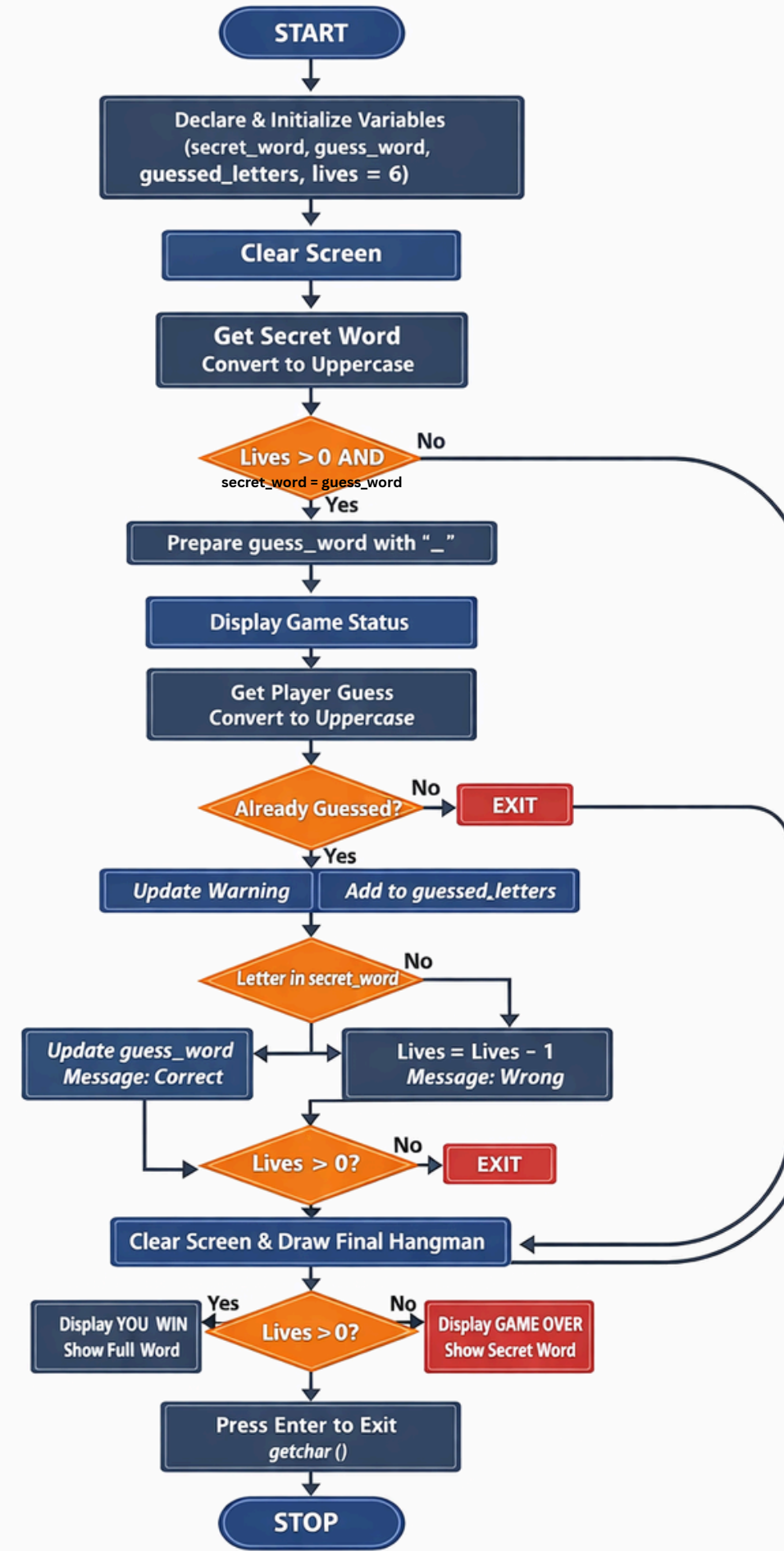
Show secret word and guessed word

Step 14: Stop

Display "Press Enter to Exit"

Use 'getchar()' to wait for Enter

FLOWCHART



CONCLUSION

Throughout the development of this Hangman application, we use several C programming concepts, including array handling, string manipulation, and user-input validation. .

- Implementing secret word masking (only for windows) to be able to play it live.
- Understood the System Buffer, and Standard Input buffer and how to work around it.
- Working with C strings
 - Store and retrieve the "secret word" from memory using the masking
 - manually inserting null terminators where required to end strings
 - using the standard library functions
 - string.h for strcmp(), strlen(), sprintf(), strchr()
 - stdio.h for printf(), scanf(), getchar(), sprintf()
 - ctype.h for isalpha(), toupper()
 - stdlib.h for exit()
 - Track and cross-reference guessed letters to prevent duplicate entries.
- Including dynamic ASCII art for visual feedback

```
+-----+  
|       |  
|       |  
|       |  
|       |  
|       |  
|       |  
|       |  
+-----+  
=====
```

WORD: _ _ _ _ _

LIVES: 6
GUESSED:

Enter a letter to guess (0 to Exit).
Input guess:

```

+-----+
|       |
|       |
|       |
|       |
|       |
|       |
|       |
|       |
|       |
|       |
+-----+

```

WORD: _ _ _ _ _

LIVES: 3

GUESSED: P L I

Sorry, 'I' is not in the word.

Input guess:

```

W      W  IIIII  N   N  N   N  EEEEE  RRRR
W      W      I   NN  N  NN  N  E      R   R
W  W  W      I   N  N  N  N  N  EEEE  RRRR
W  W  W  W      I   N   NN  N   NN  E      R   R
WW     WW  IIIII  N   N   N   N  EEEEE  R   R

WORD: BATMAN

YOU WIN! The word was: BATMAN

Press Enter to Exit...|

```

```

+-----+
|               |
|               |
|       x   x   |
|       +---+   |
|               |
|               |
|      /---\    |
|     /     \   |
|    /       \  |
|   /         \ |
|  /           \|
| /             \|
|/              \|
|=====|

```

GAME OVER! The word was: BATMAN

Press Enter to Exit...

THANK
YOU

